

Fiton Room

A mobile virtual try-on app that lets shoppers sign in with Google or Apple, upload full-body photos, submit clothing from online stores, and receive AI-generated try-on results.

Version First product version with OAuth login, authenticated history, and linked device registration.	Auth Provider Supabase Auth with Google OAuth and Sign in with Apple.
Core Backend Node.js API, Supabase Postgres, S3, SQS, worker processing, and GenAI try-on provider.	Monetization Each new user receives 5 initial Fitons; paid subscriptions grant more Fitons monthly.

Prepared for:
Fiton Room MVP planning

Document date:
May 4, 2026

Contents

1. Executive Summary
2. Product Vision
3. Version 1 User Flow
4. MVP Feature Scope
5. Recommended Tech Stack
6. Initial System Design
7. Service Connections
8. Identity and Authentication Model
9. Fitons, Trial Usage, and Monetization
10. Data Model
11. API Surface
12. Caching Strategy
13. Security and Privacy
14. Development Roadmap
15. Open Questions

1. Executive Summary

Fiton Room is a mobile virtual try-on app. A user signs in with Google or Apple, uploads one to three full-body photos, submits a clothing item from an online store, and receives an AI-generated image showing the garment on their body.

The first version should remain low-friction, but it should not rely only on an anonymous device ID. Supabase Auth provides user authentication from the start, while backend device registration remains useful for push notifications, install metadata, abuse protection, and troubleshooting.

MVP Identity

Authenticated user account through Supabase Auth using Google OAuth and Sign in with Apple.

Trial Credits

Each new user receives 5 free Fitons. A standard try-on generation consumes 1 Fiton.

Monetization

Paid users receive a monthly subscription allowance of Fitons, with one-time top-ups possible later.

Recommendation

Use Supabase Auth, Supabase Postgres, S3, SQS, Lambda or ECS/Fargate, Stripe or native IAP, and a third-party try-on API.

2. Product Vision

Fiton Room should make online clothing discovery feel more personal. The product is not just an image generator; it is a shopping companion that turns product pages, screenshots, and shared links into personal try-on previews.

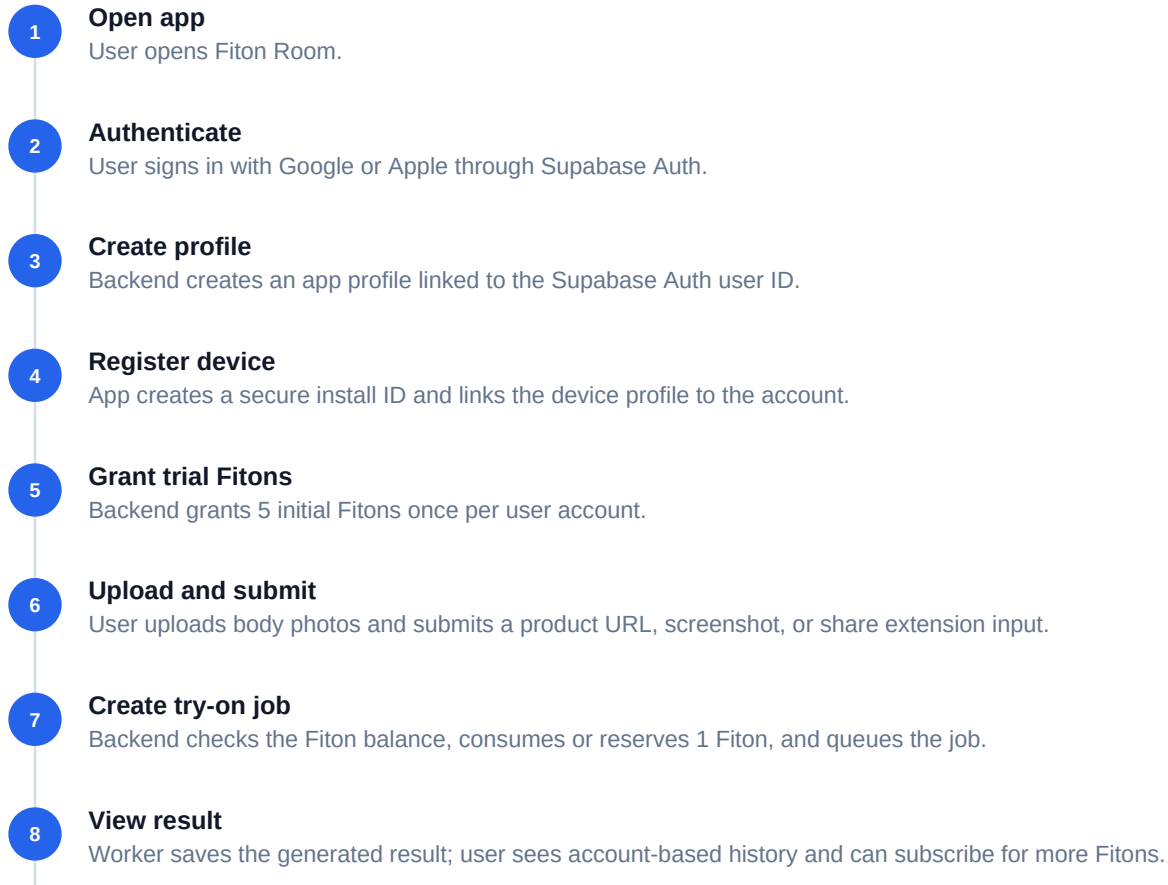
Primary MVP Outcome

Validate that users can complete the flow from authenticated sign-in to uploaded photo to useful try-on image with minimal friction and enough visual quality to make them return and consider paying.

Business Validation Goal

Learn whether 5 free Fitons are enough for users to understand the value, and whether a monthly subscription allowance is a compelling way to continue using the product.

3. Version 1 User Flow



4. MVP Feature Scope

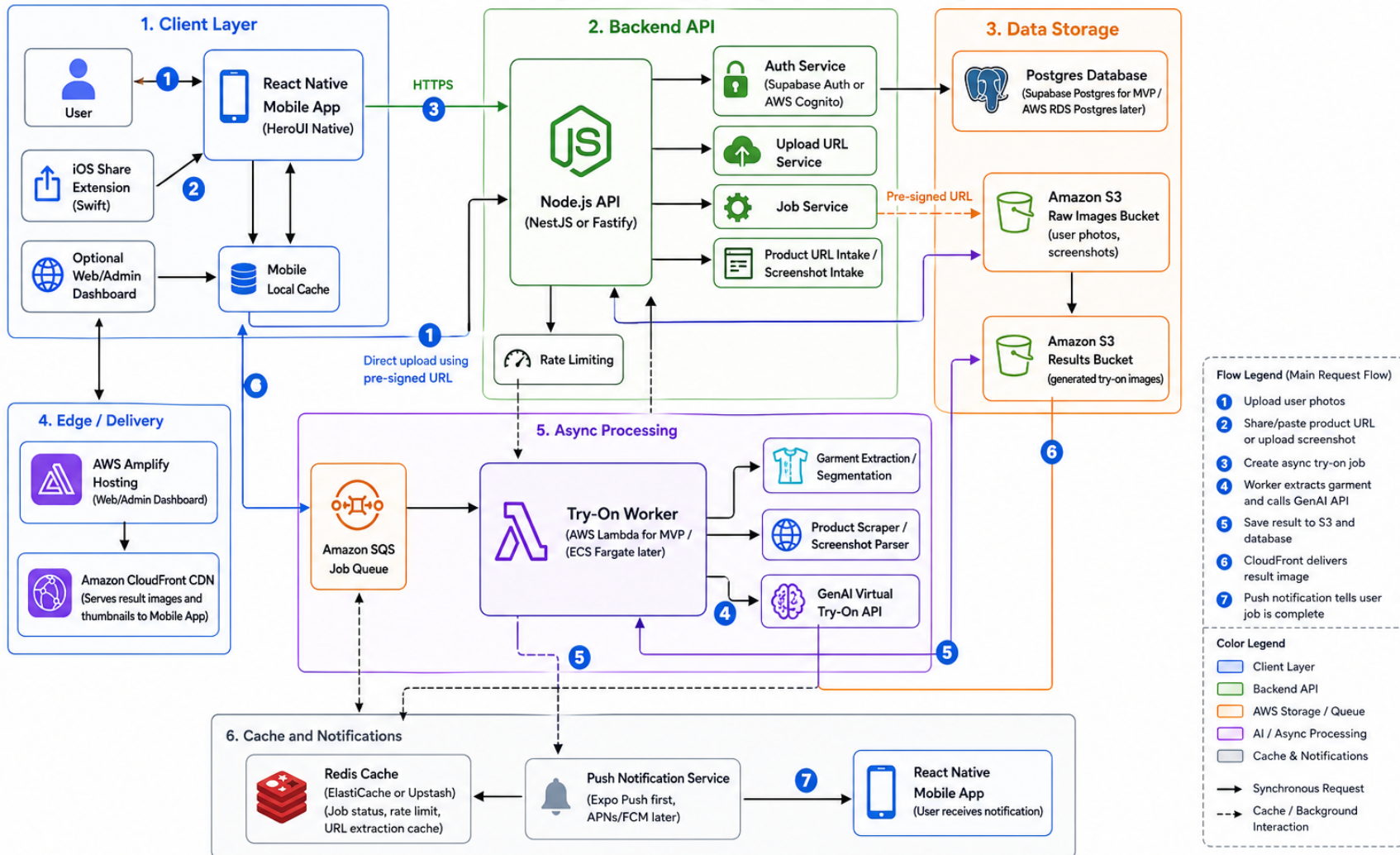
Must Have	Should Have	Later
Supabase Auth; Google OAuth; Sign in with Apple; account-based history; device registration; 5 free Fitons; balance display; Fiton-gated job creation; photo upload; URL and screenshot input; result display; deletion; S3 storage; Supabase Postgres.	iOS Share Extension; push notification on completion; CloudFront delivery; worker retry; product extraction cache; Stripe checkout or native purchase planning; admin dashboard for jobs, users, and Fiton ledger review.	Email or phone login; refined cross-device sync; Android share support; saved wardrobe; multiple variations; one-time top-ups; referral credits; affiliate links; in-house ML; mature Kubernetes/EKS scale.

5. Recommended Tech Stack

Layer	Choice	Reason
Mobile	React Native + TypeScript	Fast cross-platform mobile development.
UI	HeroUI Native	Native-focused UI layer matching the app plan.
iOS Extension	Swift / SwiftUI	Best fit for iOS share extension behavior.
Authentication	Supabase Auth	Built-in OAuth, JWT handling, user sessions, and close fit with Postgres.
OAuth Providers	Google and Apple	Simple first auth surface; Apple is important for iOS expectations and App Store compliance.
Backend API	Node.js with NestJS or Fastify	Good developer speed, strong ecosystem, and simple API work.
Database	Supabase Postgres	Fast MVP setup and direct relationship to Supabase Auth users.
Storage/CDN	Amazon S3 + CloudFront	Reliable private storage and fast result delivery.
Queue/Worker	SQS + Lambda first	Simple async orchestration; move to ECS/Fargate if jobs are long-running.
Payments	Stripe Billing or native IAP	Stripe is fastest operationally; app store rules may require native purchase flows.

6. Initial System Design

Virtual Try-On App System Design



Technology Roles

Technology	How Fiton Room uses it	Why it is in the initial design
React Native + TypeScript	Build the iOS/Android mobile app screens, account flow, upload flow, job status, gallery, balance, and subscription entry points.	One app codebase keeps MVP speed high while still supporting native mobile UX.
HeroUI Native	Provide consistent app controls, cards, buttons, lists, forms, and visual styling.	Keeps the product UI cohesive without building every primitive from scratch.
Swift / SwiftUI Share Extension	Accept product URLs or screenshots shared from Safari or shopping apps into Fiton Room on iOS.	Makes clothing capture feel native and fast for the first mobile workflow.
Supabase Auth	Handle Google OAuth, Sign in with Apple, sessions, JWTs, and auth user identity.	Avoids building auth infrastructure and gives the backend a reliable token to verify.
Node.js API	Own app-specific business logic: profile sync, device registration, pre-signed uploads, Fiton balance checks, job creation, result history, deletion, and subscription state.	Keeps client apps thin and prevents users from bypassing Fiton or ownership rules.
Supabase Postgres	Store app users, devices, photo metadata, product inputs, jobs, results, Fiton ledger entries, subscription state, payment customers, push tokens, and deletion requests.	Real relational database with fast MVP setup and close integration with Supabase Auth.
Amazon S3	Store private raw body photos, clothing screenshots, generated results, and thumbnails using object keys stored in Postgres.	Image files should not live in the database or pass through the API server.
CloudFront	Serve generated results and thumbnails quickly through controlled URLs.	Improves image load time and keeps S3 buckets private.
Amazon SQS	Queue try-on jobs after the API records the job and Fiton debit/reservation.	Separates app responsiveness from slower AI generation work.
Lambda first, ECS/Fargate later	Run worker logic that downloads inputs, extracts garment details, calls the try-on provider, writes results, updates jobs, and triggers notifications.	Lambda is simple for early async work; containers are available if jobs become long-running.
GenAI Try-On Provider	Generate the try-on image from the user's body photo and garment input.	Using a third-party provider lets the MVP validate product demand before building ML in-house.
Stripe Billing or Native IAP	Create subscriptions, handle renewals/cancellations, and trigger monthly Fiton grants through webhooks or app store purchase events.	Payments must be outside the client and reconciled server-side.
Expo Push first	Notify users when a generation completes or fails.	Fast MVP notification path; direct APNs/FCM can replace it later.

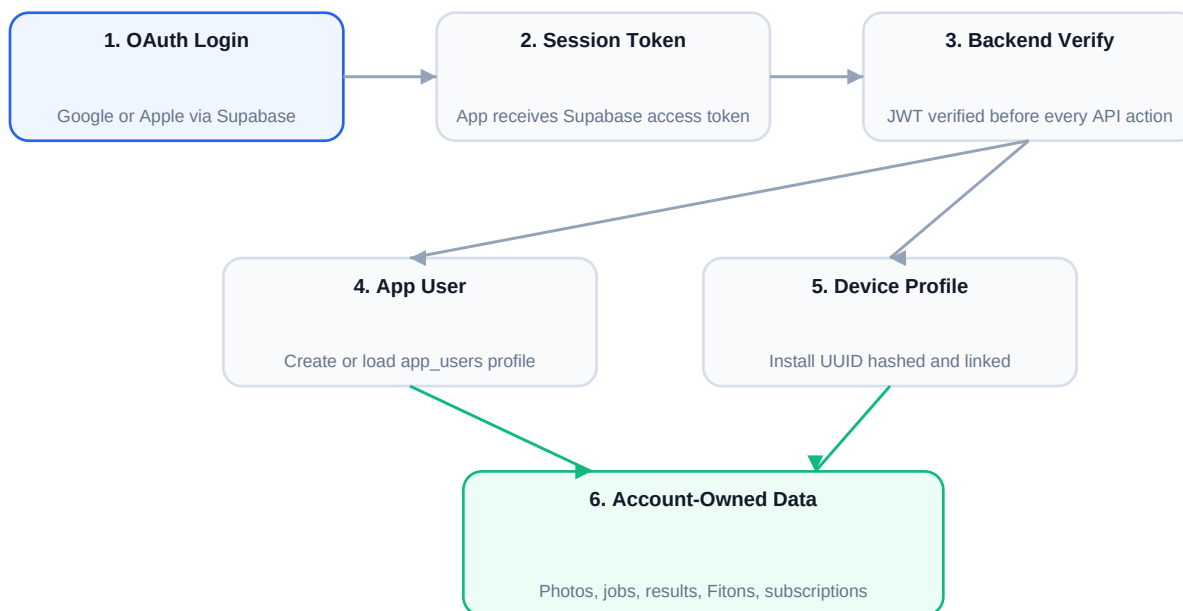
Technology	How Fiton Room uses it	Why it is in the initial design
AWS Amplify Hosting	Host the internal admin dashboard.	Simple hosting for reviewing jobs, failures, users, and Fiton ledger events.
Redis later	Add rate limits, fast job status, URL extraction cache, and high-frequency balance/status reads when usage grows.	Useful, but not required before real traffic proves the need.

7. Service Connections

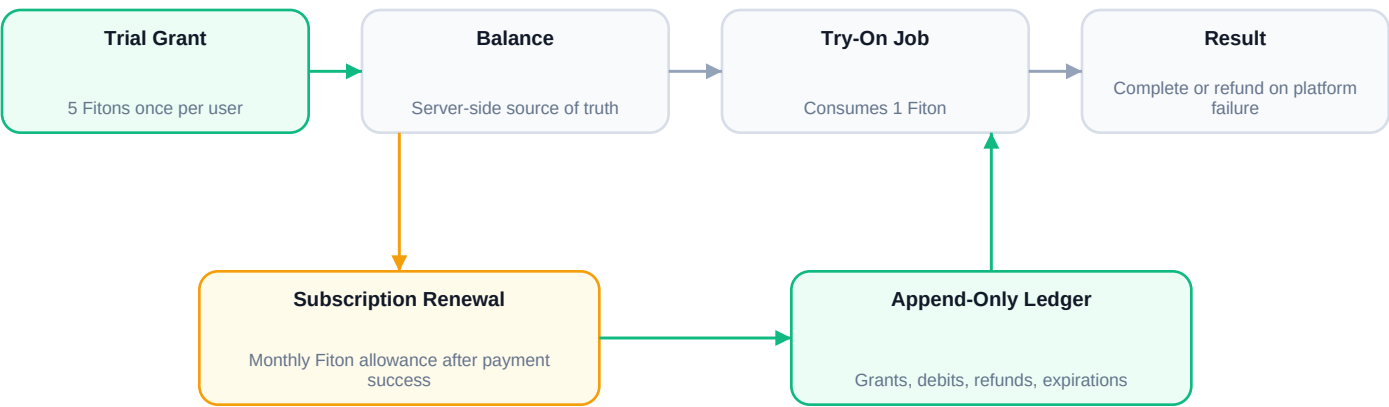
Connection	Responsibility	Important Notes
Mobile to Supabase Auth	Google OAuth and Sign in with Apple.	App receives a Supabase session and sends the access token to the backend.
Mobile to Backend API	Authenticated app operations.	Backend verifies JWT and derives the internal user server-side.
Mobile to S3	Direct photo and screenshot uploads.	Use short-lived pre-signed URLs; large files do not pass through the API.
Backend to Postgres	User profiles, jobs, results, Fitons, subscriptions, and payment mappings.	Store S3 object keys, not raw images.
Backend to SQS	Try-on job orchestration.	Store job row, record Fiton debit or reservation, then enqueue a compact job message.
Payment Provider to Backend	Subscription lifecycle and monthly grants.	Webhook processing must be signature-verified and idempotent.
Worker to Push	Completion or failure notification.	Expo Push is acceptable for MVP; APNs/FCM can replace it later.

8. Identity and Authentication Model

V1 requires account authentication using Supabase Auth with Google and Apple only. Device registration remains, but it is no longer the primary identity.



9. Fitons, Trial Usage, and Monetization



Fiton Rules

- Each new user receives 5 free Fitons after account creation.
- The free grant is one-time per authenticated user, not one-time per device.
- A standard try-on generation costs 1 Fiton.
- Failed jobs caused by platform or provider errors should refund the Fiton automatically.
- The backend, not the mobile client, is the source of truth for Fiton balance.
- All Fiton changes should be recorded in an append-only ledger.

Recommended MVP Subscription Model

Plan	Monthly Fitons	Purpose
Free Trial	5 one-time Fitons	Let users test the value before paying.
Starter	30 Fitons/month	Casual shoppers trying a few outfits weekly.
Plus	100 Fitons/month	Frequent shoppers and social users.
Pro	250 Fitons/month	Heavy users, creators, stylists, or testing-heavy users.

Recommended MVP expiration policy: paid monthly Fitons expire after 60 days, while the 5 free trial Fitons do not expire during MVP.

10. Initial Data Model

Table	Purpose	Key Fields
app_users	App-level user profile mapped to Supabase Auth.	id, supabase_user_id, email, display_name, avatar_url, created_at, last_seen_at
device_profiles	Registered app installs.	id, user_id, install_id_hash, platform, app_version, created_at, last_seen_at
user_photos	Uploaded body photos.	id, user_id, device_profile_id, s3_key, image_type, status, created_at
product_inputs	URL, screenshot, or share inputs.	id, user_id, device_profile_id, input_type, source_url, screenshot_s3_key, extraction_status
try_on_jobs	Async generation jobs.	id, user_id, device_profile_id, user_photo_id, product_input_id, status, fiton_cost, failure_reason
try_on_results	Generated image records.	id, job_id, user_id, result_s3_key, thumbnail_s3_key, provider

10. Initial Data Model, Continued

Table	Purpose	Key Fields
fiton_accounts	Current user balance summary.	id, user_id, available_balance, updated_at
fiton_ledger_entries	Append-only credit/debit history.	id, user_id, entry_type, amount, balance_after, source, reference_id, expires_at, created_at
subscription_plans	Configurable paid plan definitions.	id, name, monthly_fitons, price_amount, currency, active
user_subscriptions	User subscription state.	id, user_id, plan_id, provider, provider_subscription_id, status, current_period_start, current_period_end
payment_customers	Payment provider customer mapping.	id, user_id, provider, provider_customer_id, created_at
payment_events	Idempotent webhook processing.	id, provider, provider_event_id, event_type, processed_at
push_tokens	Notification routing.	id, user_id, device_profile_id, platform, token, enabled, updated_at
deletion_requests	Privacy and cleanup tracking.	id, user_id, status, requested_at, completed_at

11. Initial API Surface

Area	Routes
Auth and Profile	GET /auth/me; POST /auth/session-sync; PATCH /users/me; DELETE /users/me
Devices	POST /devices/register; PATCH /devices/:id; GET /devices
Fitons	GET /fitons/balance; GET /fitons/ledger; POST /fitons/grant-initial-trial
Uploads	POST /uploads/photo-url; POST /uploads/screenshot-url; POST /uploads/complete
Product Inputs	POST /product-inputs/url; POST /product-inputs/screenshot; GET /product-inputs/:id
Try-On Jobs	POST /try-on-jobs; GET /try-on-jobs/:id; GET /try-on-jobs; POST /try-on-jobs/:id/cancel
Results	GET /results; GET /results/:id; DELETE /results/:id
Payments	GET /subscription-plans; GET /subscriptions/me; POST /subscriptions/checkout; POST /subscriptions/customer-portal; webhooks for Stripe, App Store, and Google Play
Push and Privacy	POST /push-tokens; DELETE /push-tokens/:id; POST /privacy/delete-account-data; delete photo/result endpoints

Authorization Rule

All non-public API routes require a valid Supabase access token. The backend verifies the token and derives user_id server-side; it should not trust user IDs sent by the client.

12. Caching Strategy

MVP Caching

Mobile local cache for thumbnails; CloudFront cache for result images; database-backed job polling and Fiton balance reads.

Add Redis Later

Rate limits by user/device/IP; fast job status; duplicate URL extraction cache; worker progress; high-frequency balance reads.

13. Security and Privacy

Risk Area	Requirement
Authentication	Use Supabase Auth for Google and Apple login; verify JWTs on every authenticated backend request.
Authorization	Derive user_id from the verified token; never trust user IDs sent by the client.
Image Privacy	Use short-lived S3 pre-signed URLs; keep buckets private; serve approved results through CloudFront.
Device Identity	Hash the app-generated install ID; never use hardware identifiers as primary identity.
Deletion	Allow users to delete photos, results, and account data from the start.
Payments	Verify webhook signatures and make Fiton ledger writes server-side only.
Abuse	Protect free Fitons with lightweight rate limits, provider checks, device signals, and payment-required continuation.

14. Development Roadmap

Phase	Goal	Deliverables
Phase 1	Foundation	Mobile shell, Supabase Auth, Google OAuth, Sign in with Apple, backend auth verification, Postgres schema, device registration.
Phase 2	Fitons and Upload Flow	Initial 5-Fiton grant, balance screen, Fiton ledger, S3 upload flow, photo upload, product URL input, screenshot upload.
Phase 3	Input and Job Flow	Job creation, Fiton debit/reservation, status screen, refund behavior for failed jobs, result display.
Phase 4	AI Integration	Worker, product extraction, garment extraction, virtual try-on API, result saving, provider failure handling.
Phase 5	History and Notifications	Account-based gallery, cross-device history, push notifications, deletion flow, error states.
Phase 6	Monetization	Plan records, checkout or in-app purchase integration, webhook processing, monthly grants, cancellation/payment-failure handling.
Phase 7	Admin and Polish	Admin dashboard, job review, user review, Fiton ledger review, failure monitoring, launch hardening.

15. Open Questions

Area	Questions
Product	iOS only or iOS and Android? Which clothing input comes first? One image or multiple variations? Should users use anything before signing in?
Technical	Expo or bare React Native? NestJS or Fastify? Which try-on API provider? Debit or reserve Fitons at job creation? Which payment path launches first?
Business	Are 5 free Fitons enough? What should plans cost after provider testing? Should unused paid Fitons expire or roll over? Should top-ups exist at launch?